



Email Validation System

Checking E-mail Syntax and Deliverability using Python

Course: Python Programming (BES00007)

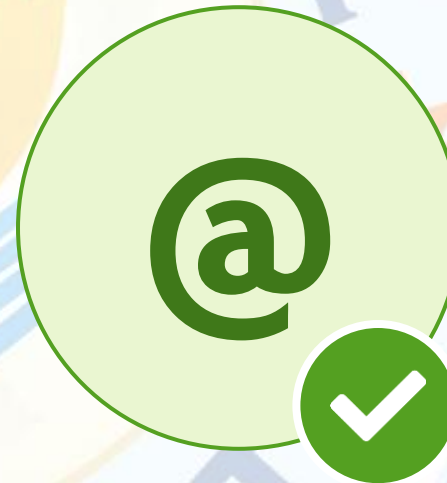
Presented by: Sayantan Bharati | BWU/BTS/25/503 | Section B

Programme: B.Tech (CSE), Batch 2025, AY 2026-27 (Odd Semester 3)

Faculty: Pranab Gharai

Department of Computer Science & Engineering, Brainware University

Date: 02-11-2026





Introduction : Why Validate E-mail Addresses?

- Every signup, contact form and mailing list collects e-mail addresses.
- A wrong address means lost communication and wasted mail-outs.
- Validation catches typos like `user@@mail.com` or `user@mail` before they are stored.
- Two levels: **syntax validation** (is it well-formed?) and **deliverability** (does the mailbox exist?).
- Python is ideal: built-in `re`, `str` methods, `smtplib`, and clean OOP.
- This project applies **strings, regex, functions, modules, exceptions and classes** together.

Before / after validation



`user@@mail.com`
malformed



`user@mail`
no TLD



`alice@mail.com`
syntactically valid



Presentation Outline

1 Anatomy of a Valid E-mail Address

2 Validation Strategies Overview

3 Regex-Based Validation

4 String Methods & Manual Parsing

5 System Design & Program Flow

6 Code Walkthrough : Object-Oriented Design

7 Advanced : DNS / MX & SMTP Checks

8 Testing & Edge Cases

9 Applications & Limitations

10 Best Practices & Conclusion



What Does a Valid Address Look Like?

- General form: **local-part @ domain.**
- **local-part**: letters, digits and `. _ % + -`; no leading/trailing dot; no two dots together.
- **domain**: dot-separated labels; each label starts/ends alphanumeric, may contain `-`.
- The final label is the TLD : at least two letters (`.com`, `.org`, `.in`).
- Length limits: local-part ≤ 64 chars, total address ≤ 254 chars.
- Governed by RFC 5321 / RFC 5322 (the Internet e-mail standards).

Anatomy of `alice.smith+news@mail.example.co.in`

`alice.smith+news`

local-part (≤ 64 chars)

@

at-sign

`mail.example`

domain labels

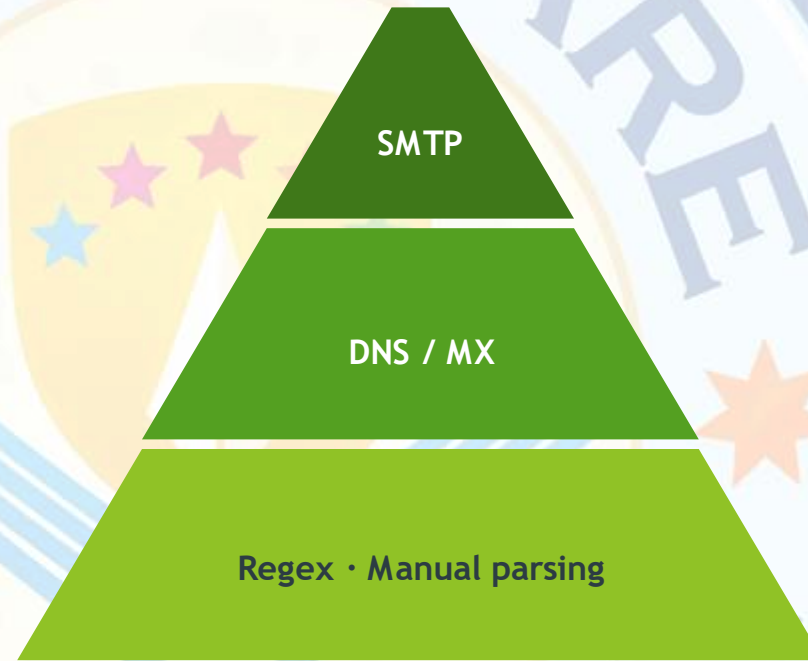
`.co.in`

TLD (≥ 2 letters)

whole address ≤ 254 characters

Three Ways to Validate

- **1. Regex** : a pattern match; fast, no network, catches most malformed input.
- **2. Manual parsing** : `str.split`, `partition`, checks on letters/digits; explicit but verbose.
- **3. Library / DNS / SMTP** : `email_validator`, MX lookup with `dnspython`, `smtplib` handshake.
- Syntax checks are **offline**; deliverability checks need the **network**.
- Best design: **layered** : cheap syntax check first, expensive network check only if it passes.
- Trade-off: strictness vs false rejections (valid-but-unusual addresses exist).



Top: slow, needs the network
Base: instant, works offline

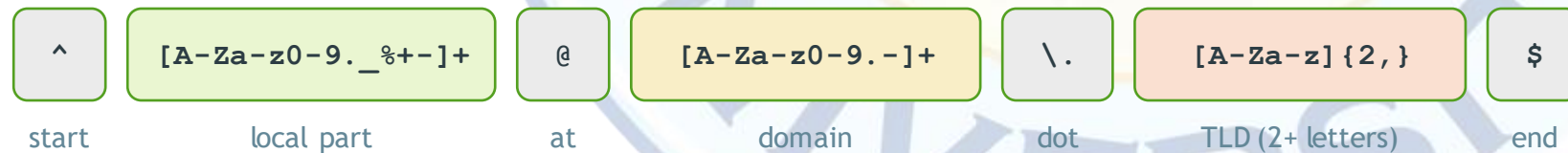
Practical Validation with re

- A **regular expression** describes the allowed character pattern.
- A good practical pattern: `^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`.
- `^ ... $` anchor the whole string : prevents partial matches.
- `re.fullmatch()` is even safer: the whole string must match.
- Regex is compact but **not a full RFC parser** : it cannot check MX records.
- Precompile the pattern for speed when validating many addresses.

```
regex_validation.py

import re
EMAIL_RE = re.compile(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")

def is_valid_syntax(email: str) -> bool:
    return bool(EMAIL_RE.fullmatch(email.strip()))
```



Alternative : Manual String Validation

- Sometimes clearer than regex: split and inspect each part yourself.
- Useful methods: `str.strip`, `str.count`, `str.split`, `str.partition`, `str.isalnum`, `str.startswith`.
- Steps: exactly one @ → non-empty local-part → valid domain labels.
- Easy to give a **specific error message** per failure (better UX).
- More code, but transparent and debuggable.
- A good teaching contrast to regex.

```
def manual_check(email: str):  
    email = email.strip()  
    if email.count("@") != 1:  
        return False, "must contain exactly one @"  
    local, domain = email.split("@")  
    if not local or local.startswith(".") or ".." in local:  
        return False, "bad local-part"  
    if "." not in domain:  
        return False, "domain needs a dot"  
    return True, "ok"
```

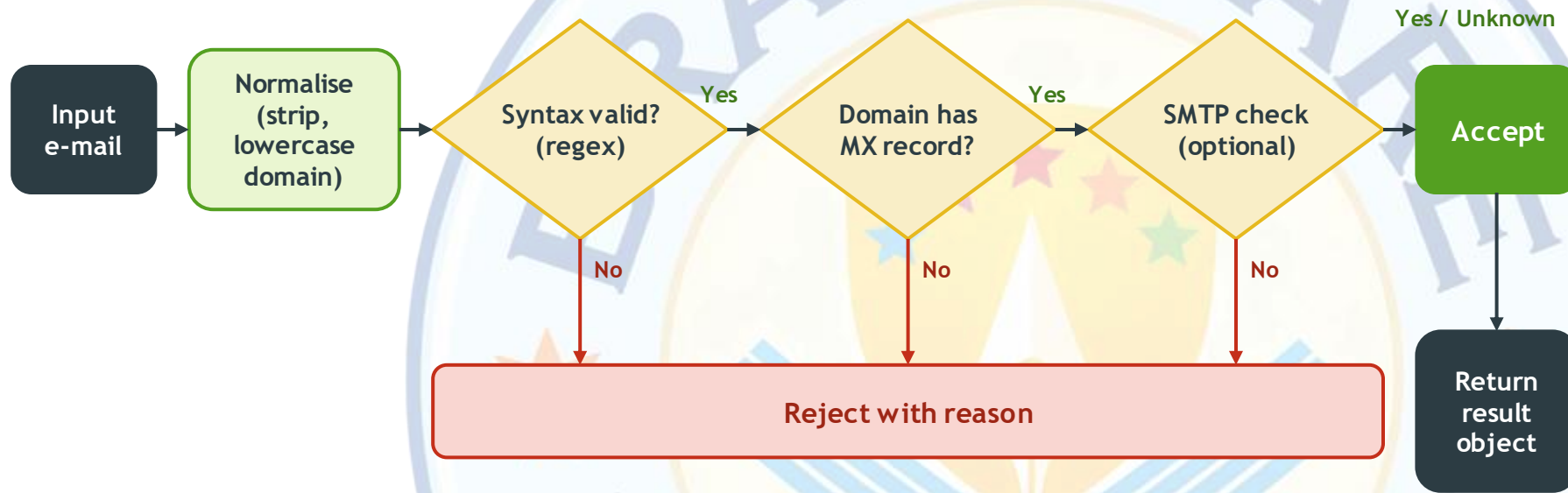
1. Exactly one @ ?
no → "must contain exactly one @"

2. Clean local-part ?
no → "bad local-part"

3. Domain has a dot ?
no → "domain needs a dot"

True, "ok"

System Design & Flow



- Pipeline: input → normalise → syntax check → domain check → deliverability → result.
- **Normalise** first: strip whitespace, lowercase the domain.
- Reject early on cheap checks; only then do network checks.
- Wrap risky steps (network, file I/O) in **try/except**.
- Return a clear result object: `{valid, reason, suggestion}`.
- Modular = each check is one small, testable function.

Code Walkthrough : A Clean Class Design

- Encapsulate the rules in a class : cohesive and reusable.
- One method per check: `normalise`, `check_syntax`, `check_domain`, `validate`.
- `validate()` returns a `ValidationResult` (dataclass): valid flag + message.
- Uses **modules** (`re`, `dataclasses`), **functions**, **classes** and **exceptions**.
- Easy to extend (add a new check without touching the others).
- Demonstrates encapsulation from the OOP module.

```
email_validator.py

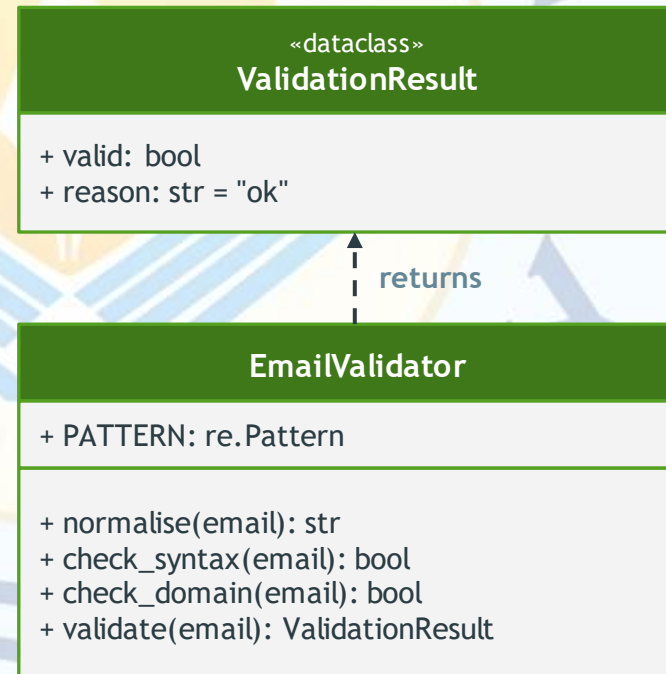
from dataclasses import dataclass
import re

@dataclass
class ValidationResult:
    valid: bool
    reason: str = "ok"

class EmailValidator:
    PATTERN = re.compile(
        r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
    )

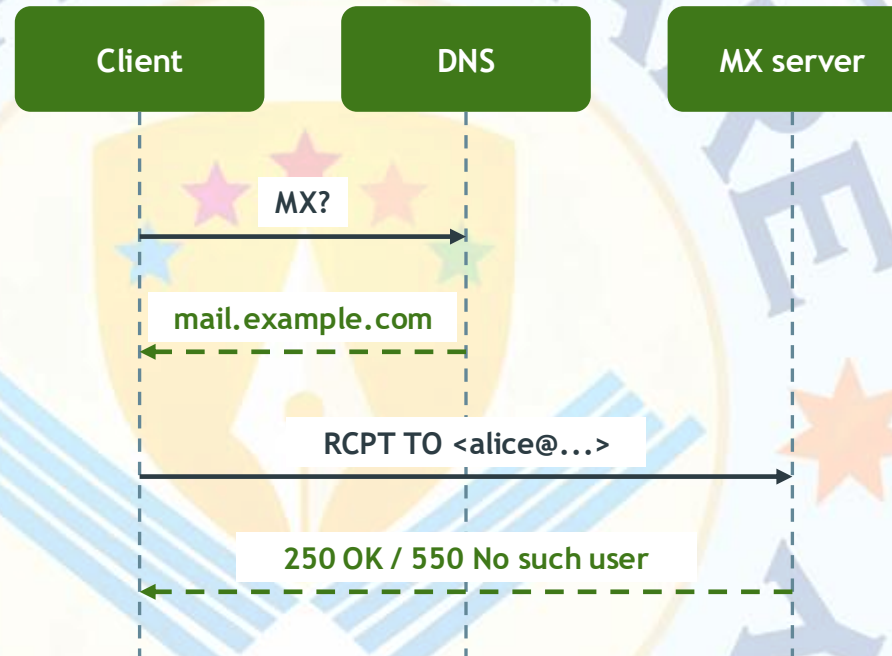
    def normalise(self, email: str) -> str:
        local, _, domain = email.strip().partition("@")
        return f"{local}@{domain.lower()}"

    def validate(self, email: str) -> ValidationResult:
        email = self.normalise(email)
        if not self.PATTERN.fullmatch(email):
            return ValidationResult(False, "invalid syntax")
        return ValidationResult(True)
```



Advanced : Is the Mailbox Real?

- Syntax-valid does not mean the domain exists or accepts mail.
- **MX record lookup** (DNS) checks the domain can receive mail : `dnspython`.
- **SMTP handshake**: connect to the MX server and issue `RCPT TO` (often blocked/risky).
- Detect **disposable** domains and **role** addresses (`info@`, `noreply@`) with a list.
- Always honour timeouts and treat “unknown” as **not invalid**.



Timeout / no reply → “unknown”
Treat as not invalid : never reject on silence.

Testing & Edge Cases

- Unit-test each rule with valid and invalid samples (`unittest` / `pytest`).
- Valid oddities: `user+tag@mail.com`, `a.b@sub.example.co.uk`, uppercase domain.
- Invalid cases: `user@@mail.com`, `user@mail`, `.user@mail.com`, `user..name@mail.com`, spaces.
- Also test empty input, very long strings, and unicode domains.
- Assert on **both** the boolean result and the **reason message**.
- Testing proves the validator is neither too strict nor too loose.

E-mail address	Result	Reason
<code>alice@mail.com</code>	Valid	ok
<code>user+tag@mail.com</code>	Valid	ok
<code>a.b@sub.example.co.uk</code>	Valid	ok
<code>ALICE@MAIL.COM</code>	Valid	ok
<code>user@@mail.com</code>	Invalid	must contain exactly one @
<code>user@mail</code>	Invalid	domain needs a dot
<code>.user@mail.com</code>	Invalid	bad local-part
<code>user..name@mail.com</code>	Invalid	bad local-part

Results shown are from running `manual_check()` on each address.

Applications & Limitations

- **Applications:** signup/login forms, newsletters, CRM data cleaning, bulk import filters.
- Catches typos early and reduces bounced mail.
- **Limitations:** regex alone cannot confirm a mailbox exists.
- Over-strict rules reject legitimate addresses : keep the pattern reasonable.
- SMTP checks can be slow, blocked, or flagged as probing.
- Privacy: an address is personal data : handle and store it responsibly.



Web forms



Newsletters



CRM cleaning



Bulk import

Watch-outs

Too strict → false rejections

SMTP probing → slow or blocked

Privacy → personal data



Best Practices & Conclusion

- ✓ Layer the checks : cheap first, network last; fail fast.
- ✓ Prefer `re.fullmatch` and precompiled patterns for performance.
- ✓ Give clear error messages; guide the user to a fix.
- ✓ Normalise before comparing; cache and timeout all network calls.
- ✓ Confirm by sending a **verification e-mail** : the only true test of deliverability.

Conclusion: a small, elegant program that unites regex, strings, functions, modules, exceptions and OOP.





References & Thank You

- R. Nageswara Rao : *Core Python Programming*, Dreamtech Press.
- Gowrishankar S. & Veena A. : *Introduction to Python Programming*, CRC Press.
- Python Standard Library docs : `re`, `smtplib`, `dataclasses`
(<https://docs.python.org/3/library/>).
- RFC 5321 / RFC 5322 : Internet Message Format and SMTP standards.
- Course lecture notes : Python modules 2, 4 & 5 (Strings, Functions/Modules, Exceptions & OOP).



**Thank you for your attention :
Questions are welcome.**

Sayantan Bharati | BWU/BTS/25/503 | Section B

